

# Відповіді на питання до захисту дипломної роботи

**Проект:** INTERIORIX — веб-система керування проєктами інтер'єру та екстер'єру

**Стек:** React, NestJS, MongoDB, TypeScript, Docker

**Призначення документа:** підготовка до усного захисту (комісія, керівник, опоненти)

У шаблоні питань згадувались «матчі», «команди», «турнірна таблиця», Spring Boot і PostgreSQL. Цей документ **адаптовано під ваш реальний проєкт INTERIORIX**: замість спортивної аналітики — CRM студії дизайну, замість Spring/PostgreSQL — NestJS/MongoDB.

## 1. Загальні питання по темі

**Яка тема вашої дипломної роботи?**

**Розробка веб-системи керування проєктами інтер'єру та екстер'єру INTERIORIX** (або формулювання з вашої записки: «Інформаційна система підтримки бізнес-процесів дизайн-студії»).

**Чому ви обрали саме цю тему?**

Тема поєднує актуальну галузь (дизайн інтер'єру та екстер'єру) з практичною потребою малого та середнього бізнесу: замість розрізнених Excel-таблиць, месенджерів і дзвінків — **єдина цифрова платформа** для клієнта і команди студії. Це дозволяє показати повний цикл розробки ПЗ: від заявки до оплати, чека та відгуку.

**У чому актуальність вашого проєкту?**

- Клієнти очікують **прозорості**: статус проєкту, кошторис, рахунки та чеки в одному кабінеті.
- Студії потребують **структурованого обліку** заявок, проєктів, команди та фінансів.
- Ринок послуг з ремонту та дизайну зростає; автоматизація зменшує помилки та час менеджерів.
- Пандемія та цифровізація прискорили перехід бізнесу в онлайн-канали комунікації з замовником.

**Яку проблему вирішує ваш веб-застосунок?**

**Розірваність процесів**: заявки в чатах, кошториси в PDF, оплати в банку, фото з об'єкта в телефоні. INTERIORIX об'єднує:

- публічний маркетинг (послуги, портфоліо, відгуки);
- **клієнтський портал** (заявки, проєкти, погодження кошторису, оплата, чеки);
- **робочий простір студії** (CRM, проєкти, заміри, фото-звіти, платежі, модерація).

## Хто є цільовою аудиторією системи?

| Аудиторія        | Хто це                   | Що робить у системі                                      |
|------------------|--------------------------|----------------------------------------------------------|
| Клієнт           | Замовник ремонту/дизайну | Подає заявку, погоджує кошторис, оплачує, залишає відгук |
| Менеджер проєкту | Співробітник студії      | Обробляє заявки, веде проєкт, кошторис, оплати           |
| Дизайнер         | Спеціаліст з дизайну     | Заміри, фото-звіти, каталог, портфоліо                   |
| Адміністратор    | Керівник / IT            | Користувачі, повний доступ, налаштування                 |
| Відвідувач сайту | Потенційний клієнт       | Переглядає послуги, портфоліо, перевіряє чек             |

## Яка головна мета дипломної роботи?

**Спроектувати та реалізувати** веб-застосунок, який автоматизує ключовий бізнес-процес дизайн-студії: **заявка** → **проєкт** → **кошторис** → **оплата** → **завершення** → **відгук**, з розмежуванням доступу за ролями.

## Які основні завдання були поставлені?

1. Проаналізувати бізнес-процеси студії дизайну.
2. Спроектувати модель даних і архітектуру системи.
3. Розробити **REST API** (серверна частина).
4. Розробити **користувацький інтерфейс** (public, portal, workspace).
5. Реалізувати авторизацію, ролі, валідацію даних.
6. Реалізувати фінансовий контур (кошторис → рахунок → mock-оплата → PDF-чек).
7. Протестувати систему та підготувати документацію.

## Що саме ви реалізували у межах дипломного проєкту?

- **Monorepo** на TypeScript: apps/backend, apps/frontend, packages/shared, packages/ui.
- **Backend**: NestJS, MongoDB, JWT, модулі CRM, проєктів, кошторисів, платежів, відгуків, файлів.
- **Frontend**: React + Vite, три контури (public / portal / workspace), React Router.
- **27 колекцій MongoDB**, ER-модель у help/tds-database.dbml.
- Завантаження файлів (референс-фото, фото-звіти).
- PDF-чеки, публічна перевірка чека /verify/:number.
- Docker Compose для локального запуску (start.bat).

## Чим ваш застосунок може бути корисний користувачам?

- **Клієнту:** один кабінет замість дзвінків «на якому етапі?»; історія заявок, кошторисів, оплат.
- **Менеджеру:** єдиний реєстр заявок і проєктів, статуси, audit log.
- **Дизайнеру:** «Моя робота», заміри, фото з об'єкта в контексті проєкту.
- **Студії:** аналітика, модерація відгуків на сайті, прозорість для замовника.

## Чи існують аналоги вашої системи? Чим ваш проєкт відрізняється?

**Аналоги (частково):** CRM для будівництва (Bitrix24, amoCRM), Trello/Notion для задач, окремі конструктори сайтів.

### Відмінності INTERIORIX:

- **Спеціалізація** під дизайн-студію: заявка → кошторис → погодження клієнтом → автоматичний рахунок.
- **Три контури в одному додатку:** маркетинг + клієнтський портал + внутрішній workspace.
- **Повний B2C-цикл** з демо-оплатою та PDF-чеком.
- **Єдиний стек TypeScript** і спільний пакет @tailored/shared для узгодженості правил на клієнті й сервері.

## 2. Питання по функціоналу

\*У шаблоні: «матчі», «команди», «турнірна таблиця». Нижче — відповідники в INTERIORIX.\*

| Питання з шаблону | Аналог у INTERIORIX                                          |
|-------------------|--------------------------------------------------------------|
| Перегляд матчів   | Списки <b>заявок і проєктів</b>                              |
| Сторінка команд   | <b>Команда студії</b> (/team) або <b>бригади</b> в workspace |
| Турнірна таблиця  | <b>Аналітика / звіти</b> workspace, дашборди                 |
| Деталі матчу      | <b>Картка заявки / проєкту</b>                               |
| Обране            | Референс-фото, портфоліо, сповіщення                         |

## Які основні функції має ваш веб-застосунок?

**Public:** головна, каталог послуг, портфоліо, відгуки, команда, контакти, перевірка чека.

**Portal (клієнт):** нова заявка, мої заявки/проєкти, кошториси, рахунки, mock-оплата, чеки, відгуки, сповіщення, профіль.

**Workspace (студія):** заявки (CRM), проєкти, кошториси, заміри, фото-звіти, платежі, чеки, модерація відгуків, користувачі (адмін), audit, аналітика.

## Як користувач взаємодіє із системою?

1. Відвідувач переглядає **публічний сайт** без входу.
2. Для заявки — **реєстрація/вхід** → форма «Нова заявка» в порталі.
3. Клієнт відстежує статус у **кабінеті**, погоджує кошторис, оплачує рахунок.
4. Співробітник входить у **workspace**, обробляє заявки, веде проєкт, змінює статуси.
5. Система надсилає **сповіщення** обом сторонам.

## Які сторінки реалізовані у застосунку?

Див. детальну таблицю в help/02-storinky-ta-kontury.md. Коротко:

- **Public:** /, /services, /portfolio, /reviews, /team, /contact, /login, /register, /verify/:number
- **Portal:** /portal/dashboard, /portal/orders, /portal/orders/new, /portal/projects, /portal/invoices, /portal/receipts, /portal/reviews, ...
- **Workspace:** /workspace/dashboard, /workspace/orders, /workspace/projects, /workspace/estimates, /workspace/measurements, /workspace/payments, /workspace/my-work, ...

## Як працює перегляд «матчів» → заявок і проєктів?

- **Клієнт:** /portal/orders — таблиця/картки заявок з кодом (TDS-LEAD-...), статусом, датою; клік → деталі заявки.
- **Менеджер:** /workspace/orders — усі заявки студії з фільтрами; можна кваліфікувати, конвертувати в проєкт.
- **Проєкти:** /portal/projects та /workspace/projects — список з етапом (DRAFT, DESIGN, IN\_PROGRESS, COMPLETED тощо).

## Як працює сторінка «команд»?

- **Public /team:** презентація спеціалістів студії для відвідувачів.
- **У проєкті:** менеджер призначає **менеджера та дизайнера** на картці проєкту; дизайнер бачить задачі в «Моя робота».

## Як реалізована «турнірна таблиця» → аналітика та звіти?

У workspace є **Огляд** (/workspace/dashboard), **Аналітика** та **Звіти** — зведення по заявках, проєктах, оплатах. Це не спортивна таблиця, а **бізнес-дашборд** для керівника та менеджера.

## Що показується на сторінці деталей «матчу» → заявки / проєкту?

**Заявка:** код, статус, опис, адреса, бюджет, референс-фото, посилання на створений проєкт.

**Проєкт:** етап, команда, заміри, фото-звіти, версії кошторису, рахунки, історія платежів, можливість залишити відгук після COMPLETED.

## Чи є можливість пошуку або фільтрації даних?

Так: списки заявок і проєктів у workspace підтримують **фільтрацію за статусом**; у формах — searchable select для послуг; публічний каталог послуг і портфоліо структуровані за slug.

## Як працює функція «обраного»?

Прямого «favorites» як у спортивному застосунку немає. Аналоги:

- **Референс-фото** при створенні заявки (upload або URL).
- **Портфоліо**: «Хочу подібний проєкт» — прив'язка до кейсу.
- **Сповіщення** — користувач бачить важливі події в /portal/notifications.

## Чи передбачена авторизація користувача?

Так: **реєстрація клієнта**, **вхід** (/login), JWT-токен після успішної автентифікації. Закриті маршрути /portal/\* та /workspace/\* недоступні без входу.

## Які ролі користувачів передбачені у системі?

CLIENT, менеджер (MANAGER), дизайнер (DESIGNER), адмін (ADMIN), також у seed — бригадир для задач на об'єкті. Права описані в help/01-biznes-logika.md.

## Які дані користувач може переглядати?

Залежить від ролі: клієнт — **лише свої** заявки, проєкти, рахунки; менеджер — заявки та проєкти студії; дизайнер — призначені проєкти та «моя робота»; адмін — усе + користувачі.

## Чи можна додавати нові «матчі» / заявки?

Так: клієнт створює **нову заявку** (/portal/orders/new). Менеджер **конвертує** заявку в проєкт. Нові позиції кошторису, заміри, фото-звіти додає команда в workspace.

## Які функції можна додати у майбутньому?

- Реальний **платіжний шлюз** (LiqPay, Stripe).
  - **Календар** виїздів і дедлайнів (API вже частково є).
  - **Електронний підпис** документів.
  - **Мобільний застосунок** для бригади на об'єкті.
  - Інтеграція з **1С / бухгалтерією**.
  - Чат клієнт–менеджер у контексті проєкту.
-

### 3. Питання по технологіях

\*У шаблоні: Spring Boot, PostgreSQL. У вашому проєкті — NestJS і MongoDB.\*

#### Які технології ви використали для розробки?

| Шар      | Технологія                                          |
|----------|-----------------------------------------------------|
| Мова     | TypeScript                                          |
| Frontend | React 18, Vite, Tailwind CSS, React Router, Zustand |
| Backend  | NestJS (Node.js), class-validator                   |
| БД       | MongoDB, Mongoose                                   |
| Auth     | JWT, bcrypt                                         |
| PDF      | PDFKit                                              |
| DevOps   | Docker Compose                                      |
| Monorepo | npm workspaces                                      |

#### Чому ви обрали саме React для frontend?

- **Компонентний підхід** — одна кодова база для public, portal і workspace.
- **SPA** — швидкі переходи без повного перезавантаження сторінки.
- Велика екосистема, зручна інтеграція з REST API.
- Можливість винести UI в пакет @tailored/ui.

#### Чому для backend використано NestJS (замість Spring Boot)?

- **Єдина мова TypeScript** з frontend і @tailored/shared — одні правила валідації та статусів.
- NestJS дає **модульну архітектуру** (CRM, projects, payments) подібно до Spring, але на Node.js.
- Швидкий старт для monorepo, добра підтримка MongoDB через Mongoose.
- \*Якщо запитують про Spring:\* «Розглядався класичний Java-стек; для дипломного MVP обрано TypeScript full-stack через цілісність кодової бази та швидкість ітерацій.»

#### Яку базу даних ви використовували?

**MongoDB** — документна СУБД.

#### Чому була обрана MongoDB (замість PostgreSQL)?

- Гнучкі **документи** для заявок і проєктів (різна кількість полів, вкладені дані).
- Швидке прототипування схем у Mongoose.

- Добре поєднується з NestJS і JSON REST API.
- \*Примітка:\* у репозиторії є слід Prisma/PostgreSQL — **не використовується** в поточному демо; експлуатаційно — MongoDB.

## Що таке REST API?

Архітектурний стиль обміну даними через **HTTP**: ресурси (заявки, проєкти) доступні за URL, операції — методами GET, POST, PATCH, DELETE. Клієнт надсилає JSON, сервер повертає JSON.

## Як frontend взаємодіє з backend?

1. React-компонент викликає функції з `lib/api.ts`.
2. `fetch` / `axios` надсилає запит на `http://localhost:4000/...` з заголовком `Authorization: Bearer <JWT>`.
3. NestJS-контролер приймає запит, сервіс виконує бізнес-логіку, Mongoose — запит до MongoDB.
4. Відповідь JSON відображається в UI.

## У якому форматі передаються дані між клієнтом і сервером?

**JSON** (`application/json`). Файли — **multipart/form-data** при upload.

## Для чого використовується React Router?

Маршрутизація SPA: `/`, `/portal/...`, `/workspace/...`, захист маршрутів за роллю, параметри (`/portal/projects/:code`).

## Яку роль виконує Vite у frontend-частині?

**Збірник і dev-сервер**: швидкий HMR під час розробки, production build у статичні файли для Docker/nginx.

## Які переваги має NestJS?

- Модулі, dependency injection, декоратори (схоже на Spring для Node.js).
- Guards для JWT і ролей, Pipes для валідації DTO.
- Swagger (`/api/docs`) з коробки.

## Як організована структура вашого проєкту?

|                                 |                                    |
|---------------------------------|------------------------------------|
| <code>apps/backend/</code>      | – NestJS API                       |
| <code>apps/frontend/</code>     | – React SPA                        |
| <code>packages/shared/</code>   | – спільні типи, валідація, статуси |
| <code>packages/ui/</code>       | – компоненти інтерфейсу            |
| <code>help/</code>              | – матеріали для диплому            |
| <code>docs/</code>              | – технічна документація            |
| <code>docker-compose.yml</code> |                                    |

## Які основні компоненти є у frontend?

Layout (public, portal, workspace), сторінки (PortalNewOrderPage, ProjectDetail, ...), SearchableSelect, PublicImage, форми intake, Zustand store для auth.

## Які основні контролери або сервіси є у backend?

AuthController, PortalController, CrmController, ProjectsController, EstimatesController, PaymentsController, FilesController, NotificationsController, PublicController, UsersController, AuditController, ... — кожен з відповідним \*Service.

## 4. Питання по базі даних

\*Замість «таблиць PostgreSQL» — **колекції MongoDB**.\*

### Які основні «таблиці» / колекції є у базі даних?

**27 колекцій**, зокрема: users, roles, client\_profiles, staff\_profiles, orders, projects, estimates, estimate\_items, invoices, payments, receipts, catalog\_services, portfolio\_items, design\_measurements, photo\_reports, reviews, notifications, audit\_logs, ...

Повний перелік — help/04-model-danyh.md та help/tds-database.dbml.

### Які сутності ви реалізували?

Користувач, роль, профіль клієнта/співробітника, **заявка, проєкт, кошторис**, рахунок, платіж, чек, заміри, фото-звіт, відгук, сповіщення, audit log.

### Які зв'язки існують між колекціями?

Через **ObjectId**: orders.clientId → client\_profiles, orders.projectId → projects, projects → estimates → estimate\_items, invoices → payments → receipts, projects → photo\_reports, тощо. ER-діаграма в dbml.

### Як зберігається інформація про «матчі» → заявки?

Колекція **orders**: код (TDS-LEAD-...), статус (NEW, QUALIFIED, CONVERTED, REJECTED), опис, адреса, телефон, бюджет, посилання на клієнта та проєкт.

### Як зберігається інформація про «команди»?

- **Public team** — контент сторінки / seed.
- **Команда проєкту** — поля managerId, designerId у projects.
- **Бригади** — колекції teams, team\_members (операційні задачі).



## Як реалізовано «турнірну таблицю» → звіти?

Не окрема таблиця в БД, а **агрегація** даних з orders, projects, payments у сервісах dashboard/reports для UI.

## Які типи даних використовуються?

ObjectId, String, Number, Date, Boolean, вкладені об'єкти та масиви (MongoDB documents).

## Що таке первинний ключ?

У MongoDB — поле `_id` типу ObjectId, унікальний ідентифікатор документа.

## Що таке зовнішній ключ?

Поля-посилання на `_id` іншої колекції, наприклад `projectId`, `clientId`, `estimateId`. MongoDB не enforce FK на рівні БД — **цілісність на рівні застосунку** (Mongoose + сервіси).

## Для чого потрібні зв'язки між колекціями?

Щоб об'єднати дані: «усі кошториси проєкту X», «усі платежі клієнта Y», «чек до платежу Z».

## Як забезпечується цілісність даних?

- Валідація DTO на backend (class-validator).
- Спільна валідація в @tailored/shared на frontend.
- Бізнес-правила в сервісах (наприклад, рахунок створюється лише після APPROVED кошторису).
- Стан-машина статусів проєкту.

## Чи використовували ви ORM?

Так — **Mongoose** (ODM для MongoDB). Аналог ORM/JPA для документної БД.

## Що таке JPA / Hibernate?

Технології Java/Spring для реляційних БД. **У вашому проєкті не використовуються** — замість них Mongoose.

## Як backend отримує дані з бази?

Сервіси NestJS інjektять Mongoose-моделі (@InjectModel) і виконують find, findOne, create, updateOne, aggregate.

---

## 5. Питання по архітектурі

---

### Яку архітектуру має ваш застосунок?

**Клієнт-серверна, тришарова:** presentation (React) → application (NestJS) → data (MongoDB).  
Monorepo з shared-пакетами.

### Чому ви розділили frontend і backend?

- Незалежна розробка UI та API.
- Можливість масштабувати сервер окремо.
- Безпека: бізнес-логіка та БД лише на сервері.
- Один backend може обслуговувати web і в майбутньому mobile.

### Які переваги має клієнт-серверна архітектура?

Централізовані дані, контроль доступу, оновлення логіки без перевстановлення застосунку у клієнта (достатньо оновити frontend).

### Як побудована взаємодія між частинами системи?

Браузер → React (Vite) → HTTP/JSON → NestJS REST → Mongoose → MongoDB

### Що таке API endpoint?

Конкретна URL-адреса + метод HTTP, наприклад GET /api/crm/orders, POST /api/portal/orders.

### Які endpoint-и реалізовані у backend?

Документовані в **Swagger**: <http://localhost:4000/api/docs>. Групи: auth, crm, projects, estimates, payments, portal, public, files, notifications, ...

### Як відбувається запит даних з frontend?

Компонент → portalApi.getOrders() → fetch з JWT → контролер → сервіс → MongoDB → JSON → state React → рендер.

### Як обробляються помилки?

HTTP-коди (400, 401, 403, 404, 500), повідомлення валідації з backend, відображення на формах (fieldErrors), toast/текст помилки в UI.

### Як система реагує, якщо даних немає?

Empty state на списках («Заявок поки немає»), 404 для неіснуючого коду проєкту, коректні повідомлення в API.

## Як можна масштабувати систему у майбутньому?

- Горизонтальне масштабування NestJS за load balancer.
  - MongoDB replica set.
  - CDN для статички frontend.
  - Окремий сервіс файлів (S3).
  - Черги (Redis/Bull) для сповіщень і PDF.
- 

## 6. Питання по тестуванню

---

### Як ви тестували свій застосунок?

- **Ручне E2E** за docs/TESTING\_GUIDE\_UK.md (заявка → кваліфікація → проєкт → кошторис → оплата).
- **Автоматизовані скрипти:** npm run smoke, npm run verify:upload, npm run build, npm run lint, npm run test.
- Перевірка в **браузері** (Chrome).
- **Swagger** для API.

### Які функції були перевірені?

Реєстрація/вхід, створення заявки, upload фото, зміна статусів, кошторис, погодження, mock-оплата, PDF-чек, verify, модерація відгуку.

### Які тестові сценарії ви використовували?

Демо-акаунти: client@tailored.demo, manager@tailored.demo, designer@tailored.demo, admin@tailored.demo (пароль Demo12345!).

### Чи перевіряли ви роботу frontend?

Так — всі три контури, форми, валідація, адаптивність.

### Чи перевіряли ви роботу backend?

Так — через Swagger і E2E-сценарії.

### Чи тестували ви API?

Так — Swagger UI, smoke-скрипти, verify-upload end-to-end.

### Які помилки виникали під час розробки?

- Upload файлів (кириличні імена, прийняття лише https URL).
- Прев'ю фото (aspect ratio).

- Розбіжність валідації client/server — вирішено через @tailored/shared.

### Як ви їх виправляли?

Систематичний debugging, логи Docker, виправлення ланцюжка upload → filelds → photo-reports, спільні валідатори.

### Чи перевіряли ви застосунок у браузері?

Так, основний спосіб приймання для UI.

### Чи перевіряли ви коректність даних у базі?

Так — через MongoDB seed, перевірка записів після E2E, npm run seed:report.

### Що показали результати тестування?

Система проходить **повний бізнес-цикл** у демо-режимі; критичні сценарії працездатні.

### Чи можна вважати систему працездатною?

Так, як **навчальний MVP** з mock-оплатою — готовий до демонстрації на захисті.

---

## 7. Питання по безпеці

---

### Як у вашій системі реалізована авторизація?

POST /auth/login → перевірка email/пароля → **JWT** у відповіді → frontend зберігає токен → кожен запит з Authorization: Bearer ... → **JwtAuthGuard** на backend.

### Як захищаються дані користувача?

HTTPS у production; JWT з терміном дії; доступ до чужих заявок/проектів заборонений на рівні сервісу (фільтр по clientId).

### Чи використовується шифрування паролів?

Так — **bcrypt** (хеш, не plaintext).

### Чому не можна зберігати пароль у відкритому вигляді?

При витоку БД зловмисник отримує всі паролі; хеш дозволяє лише перевірку при вході, не відновлення пароля напямую.

### Що таке JWT?

JSON Web Token — підписаний токен з payload (userId, role); сервер перевіряє підпис без збереження сесії в БД.

## Для чого потрібна валідація даних?

Захист від некоректного вводу, XSS через поля, порушення бізнес-правил (наприклад, дата в минулому, невалідний телефон).

## Як система захищена від некоректного введення?

class-validator на DTO, @tailored/shared на формах, sanitization URL для фото, ліміти розміру файлів.

## Які потенційні вразливості можуть бути у веб-застосунку?

CSRF (менше актуально для JWT API), XSS, brute-force login, завантаження шкідливих файлів — частково пом'якшено валідацією типів файлів.

## Як можна покращити безпеку у майбутньому?

Rate limiting, refresh tokens, 2FA, antivirus scan upload, OWASP audit, secrets у vault, production HTTPS only.

## Чи передбачена перевірка прав доступу?

Так — **RolesGuard**, перевірка ролі на endpoint; клієнт не може викликати workspace API.

---

## 8. Питання по інтерфейсу

---

### Які принципи дизайну ви використовували?

Простота, **consistency** через @tailored/ui, зрозуміла ієрархія (заголовки, картки), mobile-friendly (Tailwind responsive).

### Чому було обрано саме таку кольорову палітру?

Стиль **INTERIORIX** — сучасний, «студійний»: нейтральні slate-відтінки, акценти для СТА; асоціація з дизайном інтер'єру, не агресивна палітра.

### Чи зручний інтерфейс для користувача?

Так — окремі меню для клієнта і студії, статуси українською, підказки в формах (наприклад, телефон UA + міжнародний).

### Як користувач переходить між сторінками?

React Router, бічне меню portal/workspace, header на public, breadcrumbs на деталях.

### Чи адаптований сайт під різні екрани?

Так — Tailwind breakpoints, responsive grid, мобільне меню.

### Як відображаються таблиці, «матчі», «команди»?

Таблиці/картки зі статус-badges, кольорове кодування етапів проєкту, прев'ю фото з object-contain.

### Чи є повідомлення про помилки або завантаження?

Так — fieldErrors, текст під полями, стани loading на кнопках, сповіщення в центрі notifications.

### Як ви покращили user experience?

Єдиний вхід, демо-кнопки логінів, searchable select, миттєве прев'ю upload, українські підписи статусів з @tailored/shared.

### Які елементи інтерфейсу є найважливішими?

Форма **нової заявки**, **картка проєкту**, блок **кошторису** з погодженням, **оплата**, навігація portal/workspace.

### Що б ви покращили у дизайні в майбутньому?

Темна тема, більше анімацій переходів, онбординг для нового клієнта, кращі графіки в аналітиці.

## 9. Питання по практичній частині (демонстрація)

| Що показати        | Дія                                                   |
|--------------------|-------------------------------------------------------|
| Запуск             | start.bat → http://localhost:3000                     |
| Головна            | / — послуги, СТА «Створити заявку»                    |
| «Матчі»            | /portal/orders або /workspace/orders                  |
| «Турнірна таблиця» | /workspace/dashboard або analytics                    |
| «Команда»          | /team або призначення на проєкті                      |
| Фільтрація         | Фільтр статусів у workspace orders                    |
| Frontend ↔ Backend | DevTools Network або Swagger                          |
| БД                 | help/tds-database.dbml на dbdiagram.io                |
| API-запит          | Swagger: POST /auth/login, GET /crm/orders            |
| Код                | validatePortalOrderIntake, ProjectsService, JWT guard |

### Приклад API-запиту (вхід)

```
POST /api/auth/login
Content-Type: application/json
```

```
{
  "email": "client@tailored.demo",
  "password": "Demo12345!"
}
```

Відповідь містить accessToken для наступних запитів.

### Важливий фрагмент коду

Спільна валідація заявки в packages/shared/src/order-intake.ts — одна логіка на frontend і в бізнес-перевірках, плюс @IsContactPhone() на backend для телефону (UA + міжнародний).

---

## 10. Питання «на подумати»

---

### Що було найскладнішим у розробці?

Координація **повного B2C-ланцюжка** (багато сутностей і статусів), upload файлів end-to-end, узгодження правил між portal і workspace.

### Яку найбільшу проблему ви вирішили?

**Розірваність процесів** — об'єднання заявки, проєкту, кошторису, оплати та чека в одній системі з прозорістю для клієнта.

### Що б ви зробили інакше, якби мали більше часу?

Реальний платіжний шлюз, більше unit/e2e тестів (Playwright), повний календар/Kanban в UI.

### Які обмеження має ваш проєкт?

Mock-оплата, локальне розгортання, без production hardening, частина legacy API (map/calendar) лише redirect.

### Чи можна використовувати систему в реальних умовах?

Як **MVP для невеликої студії** — так, після підключення реальних платежів, hosting, backup БД.

### Як можна монетизувати або розвивати застосунок?

SaaS для студій дизайну, white-label, модулі за підпискою (аналітика, 1C, SMS).

### Які функції ви б додали наступними?

Платіжний шлюз, чат, мобільний застосунок, KPI дашборд, AI-підказки для кошторису.

**Чи можна додати «інші ліги» → інші типи бізнесу?**

Так — архітектура дозволяє розширити каталог послуг, шаблони проєктів для офісів, retail, HoReCa (y seed вже є різні демо-клієнти).

**Чи можна підключити реальні зовнішні API?**

Так — платежі, SMS, Google Maps для адрес, e-sign, хмарне сховище файлів.

**Чому проєкт можна вважати завершеним MVP?**

Реалізовано **повний цикл** від заявки до відгуку, три контури, ролі, БД, API, документація, демо-дані, Docker-запуск.

---

**11. Найімовірніші питання (швидкі відповіді)**

---

**Чому ви обрали тему (не спорт, а дизайн-студія)?**

Актуальність цифровізації малого бізнесу в сфері ремонту та дизайну; можливість показати повний ERP-подібний цикл для B2C.

**Які основні функції?**

Public маркетинг, клієнтський портал (заявки, проєкти, оплата), workspace CRM (заявки, проєкти, кошториси, фото, audit).

**Які технології і чому?**

React + NestJS + MongoDB + TypeScript — full-stack monorepo, швидка розробка, гнучка модель даних.

**Як frontend взаємодіє з backend?**

REST API, JSON, JWT у заголовку Authorization.

**Яка структура бази даних?**

27 колекцій MongoDB; головний ланцюжок: users → client\_profiles → orders → projects → estimates → invoices → payments → receipts.

**Як реалізована «турнірна таблиця»?**

Dashboard і analytics у workspace — агрегація заявок, проєктів, оплат.

**Як тестували?**

E2E сценарій за інструкцією, smoke/build/lint, Swagger, браузер.



Які помилки були?

Upload, валідація URL фото, синхронізація client/server — виправлено.

Що є результатом роботи?

Працездатний веб-застосунок INTERIORIX з документацією та ER-моделлю.

Перспективи?

Реальні платежі, mobile, інтеграції, SaaS для студій.

12. Дуже короткі відповіді (запам'ятати)

**Актуальність:** студії дизайну та клієнти потребують єдиного цифрового каналу для заявок, проєктів, кошторисів, оплат і прозорого статусу робіт.

**Мета:** розробити веб-систему INTERIORIX для автоматизації бізнес-процесів дизайн-студії від заявки до відгуку.

**Що реалізовано:** frontend (React), backend (NestJS), MongoDB, REST API, public + portal + workspace, кошториси, mock-оплата, PDF-чеки, ролі, audit.

**Технології:** React, Vite, NestJS, MongoDB, TypeScript, JWT, Docker Compose; обмін — REST JSON.

**Найскладніше:** повний ланцюжок заявка → проєкт → кошторис → оплата з узгодженими статусами та правами доступу.

**Перспективи:** реальний платіжний шлюз, мобільний застосунок, аналітика, інтеграція з бухгалтерією, SaaS-модель.

Додаток: шпаргалка для демо на захисті

| Крок | Хто      | URL                                        |
|------|----------|--------------------------------------------|
| 1    | Клієнт   | /portal/orders/new — нова заявка           |
| 2    | Менеджер | /workspace/orders — кваліфікація → проєкт  |
| 3    | Менеджер | /workspace/projects/:id — кошторис SENT    |
| 4    | Клієнт   | /portal/projects/:code — погодити кошторис |
| 5    | Клієнт   | /portal/invoices — mock-оплата             |
| 6    | Будь-хто | /verify/RCPT-... — перевірка чека          |

**Логіни:** client@tailored.demo / manager@tailored.demo / пароль Demo12345!

**Swagger:** <http://localhost:4000/api/docs>

**ER-діаграма:** [help/tds-database.dbml](#) → [dbdiagram.io](#)

---

\*Документ підготовлено для проєкту INTERIORIX. Оновлюйте формулювання теми відповідно до офіційного наказу кафедри.\*